

ACADEMIC COURSE

Neural Networks

From Perceptrons to Deep Learning

A Comprehensive Guide to Artificial Neural Networks, Multilayer Perceptrons, and
Convolutional Neural Networks



Theory & Practice



TensorFlow & PyTorch



Numerical Examples

Learning Path

01

Artificial Neural Networks

Foundation Concepts

- The Perceptron Model
- Activation Functions
- Numerical Examples

</> Python Implementation

02

Multilayer Perceptron

Deep Learning Architecture

- MLP Architecture
- Forward Propagation
- Backpropagation Algorithm
- Training & Optimization

 Complete Examples

03

Convolutional Neural Networks

Computer Vision

- Convolution Operation
- Pooling Layers
- Feature Extraction
- Complete CNN Architecture

 Image Classification



Learning Approach

Theory → Mathematical Formulation → Numerical Example → Code Implementation

TensorFlow

PyTorch

CHAPTER 01

Artificial Neural Networks

Building the Foundation: From Biological Inspiration to Mathematical Models



Perceptron Model



Activation Functions



Numerical Examples

The Perceptron: Birth of Neural Computing



What is a Perceptron?

The **perceptron**, introduced by Frank Rosenblatt in 1958, is the simplest form of artificial neural network. It mimics the behavior of a biological neuron, receiving multiple inputs, processing them, and producing a single output.

Mathematical Model

Weighted Sum:

$$z = \sum (w_i \times x_i) + b = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

Activation (Step Function):

$$\hat{y} = 1 \text{ if } z \geq \theta, \text{ else } 0$$



Perceptron Learning Rule

Weight Update Formula:

$$w_i(\text{new}) = w_i(\text{old}) + \eta \times (y - \hat{y}) \times x_i$$

where η = learning rate, y = target, $\hat{y}$ = prediction

If $y = 1, \hat{y} = 0$
Increase weights

If $y = 0, \hat{y} = 1$
Decrease weights



Numerical Example: AND Gate

Train a perceptron to learn AND function with $\eta = 0.5$

Initial:

$$w_1 = 0.3, w_2 = 0.4$$

$$b = -0.5, \eta = 0.5$$

Training Data:

$$(0,0) \rightarrow 0, (0,1) \rightarrow 0$$

$$(1,0) \rightarrow 0, (1,1) \rightarrow 1$$

Epoch 1 - Sample (1,1) $\rightarrow$ 1:

$$z = 0.3 \times 1 + 0.4 \times 1 + (-0.5) = 0.2$$

$$\hat{y} = 1 \text{ (since } 0.2 \geq 0) \checkmark \text{ Correct!}$$

No weight update needed

Epoch 1 - Sample (0,1) $\rightarrow$ 0:

$$z = 0.3 \times 0 + 0.4 \times 1 + (-0.5) = -0.1$$

$$\hat{y} = 0 \text{ (since } -0.1 < 0) \checkmark \text{ Correct!}$$

</>TensorFlow

```
import tensorflow as tf
model = tf.keras.Sequential([
    tf.keras.layers.Dense(
        1, activation='sigmoid',
        input_shape=(2,))
])
model.compile(optimizer=
tf.keras.optimizers.SGD(
    learning_rate=0.5),
    loss='binary_crossentropy')
```

</>PyTorch

```
import torch
import torch.nn as nn
model = nn.Sequential(
    nn.Linear(2, 1),
    nn.Sigmoid()
)
criterion = nn.BCELoss()
optimizer = torch.optim.SGD(
    model.parameters(), lr=0.5)
```

```
class MyDenseLayer(tf.keras.layers.Layer):
```

```
    def __init__(self, output_dim):
        super(MyDenseLayer, self).__init__()
        self.output_dim = output_dim

    def build(self, input_shape):
        self.W = self.add_weight(
            shape=(input_shape[-1], self.output_dim),
            initializer="random_normal",
            trainable=True
        )
```

```
        self.b = self.add_weight(
            shape=(1, self.output_dim),
            initializer="zeros",
            trainable=True
        )
```

```
    def call(self, inputs):
        z = tf.matmul(inputs, self.W) + self.b
        return tf.math.sigmoid(z)
```

```
x = tf.constant([[1.0, 2.0, 3.0],
                 [4.0, 5.0, 6.0]])
```

```
layer = MyDenseLayer(output_dim=2)
output = layer(x)
```

```
print(output)
```

```
tf.Tensor(
[[0.4694386 0.5927551]]
```

input_dim → Number of input features

output_dim → Number of neurons in this layer

self.W → Weight matrix of shape (input_dim, output_dim)

self.b → Bias vector of shape (1, output_dim)

initializer='random_normal' → Initializes weights randomly

initializer='zeros' → Bias initialized as zero

trainable=True → Parameters will update during training

tf.matmul(inputs, self.W) → Performs matrix multiplication ($W \times X$)

self.b → Adds bias term

tf.math.sigmoid(z) → Applies Sigmoid activation function

This computes: $\text{output} = \sigma(WX + b)$

Tensor In Tensorflow

Tensors are multi-dimensional arrays used to represent data in TensorFlow

Scaler	Vector	Matrix	Tensor
1	$\begin{bmatrix} 7 \\ 8 \end{bmatrix}$	$\begin{bmatrix} 7 & 2 \\ 8 & 4 \end{bmatrix}$	$\begin{bmatrix} \begin{bmatrix} 7 & 2 \end{bmatrix} & \begin{bmatrix} 2 & 7 \end{bmatrix} \\ \begin{bmatrix} 8 & 4 \end{bmatrix} & \begin{bmatrix} 4 & 8 \end{bmatrix} \end{bmatrix}$

Step 1: Given Input

You passed:

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

Shape = (2,3)

Step 2: Assume Random Weights (Example)

Since TensorFlow initializes randomly, let's assume your weights looked something like:

$$W = \begin{bmatrix} 0.10 & -0.20 \\ 0.05 & 0.30 \\ -0.10 & 0.15 \end{bmatrix}$$

Shape = (3,2)

Bias:

$$b = \begin{bmatrix} 0 & 0 \end{bmatrix}$$

For each output neuron j :

$$y_j = \sigma \left(\sum_{i=1}^3 x_i w_{ij} + b_j \right)$$

Step 3: Matrix Multiplication (WX)

For first input: $[1, 2, 3]$

First neuron:

$$z_1 = (1)(0.10) + (2)(0.05) + (3)(-0.10)$$

$$z_1 = 0.10 + 0.10 - 0.30 = -0.10$$

Second neuron:

$$z_2 = (1)(-0.20) + (2)(0.30) + (3)(0.15)$$

$$z_2 = -0.20 + 0.60 + 0.45 = 0.85$$

So before activation:

$$z = [-0.10, 0.85]$$

Step 4: Apply Sigmoid

Sigmoid formula:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

For -0.10:

$$\begin{aligned}\sigma(-0.10) &= \frac{1}{1 + e^{0.10}} \\ &= \frac{1}{1 + 1.105} \\ &= 0.475\end{aligned}$$

For 0.85:

$$\begin{aligned}\sigma(0.85) &= \frac{1}{1 + e^{-0.85}} \\ &= \frac{1}{1 + 0.427} \\ &= 0.700\end{aligned}$$

So output becomes:

[0.475, 0.700]

Step 5: Same for Second Input [4,5,6]

First neuron:

$$\begin{aligned}(4)(0.10) + (5)(0.05) + (6)(-0.10) \\ = 0.40 + 0.25 - 0.60 \\ = 0.05\end{aligned}$$

Second neuron:

$$\begin{aligned}(4)(-0.20) + (5)(0.30) + (6)(0.15) \\ = -0.80 + 1.50 + 0.90 \\ = 1.60\end{aligned}$$

Apply sigmoid:

$$\sigma(0.05) = 0.512$$

$$\sigma(1.60) = 0.832$$

So second row becomes:

[0.512, 0.832]



Final Output Matrix

$$\begin{bmatrix} 0.475 & 0.700 \\ 0.512 & 0.832 \end{bmatrix}$$

CHAPTER 02

Multilayer Perceptron

Going Deeper: Architecture, Forward Propagation, and Backpropagation



MLP Architecture



Forward Propagation



Backpropagation

Copyright © All Rights Reserved Doran

MLP Architecture and Forward Propagation



MLP Architecture

A **Multilayer Perceptron (MLP)** is a feedforward neural network with at least three layers: input, hidden, and output. It can learn non-linear relationships through hidden layers.

I Input Layer

Receives raw features. No computation, just passes data to hidden layer.

H Hidden Layer(s)

Performs non-linear transformations. Can have multiple hidden layers (deep network).

O Output Layer

Produces final predictions. Activation depends on task (sigmoid, softmax, linear).

Universal Approximation Theorem

An MLP with at least one hidden layer and a non-linear activation function can approximate any **continuous function** to arbitrary precision, given sufficient neurons.

→ Forward Propagation

For each layer l , compute:

$$\mathbf{z}[l] = \mathbf{W}[l] \cdot \mathbf{a}[l-1] + \mathbf{b}[l]$$

$$\mathbf{a}[l] = g(\mathbf{z}[l])$$

where g = activation function

Numerical Example:

2-2-1 MLP, inputs [0.5, 0.3]

Layer 1 (Hidden):

$$z_1 = 0.2 \times 0.5 + 0.4 \times 0.3 + 0.1 = 0.32$$

$$a_1 = \text{ReLU}(0.32) = 0.32$$

$$z_2 = 0.1 \times 0.5 + 0.3 \times 0.3 + 0.2 = 0.34$$

$$a_2 = \text{ReLU}(0.34) = 0.34$$

Layer 2 (Output):

$$z = 0.5 \times 0.32 + 0.6 \times 0.34 + 0.1 = 0.434$$

$$\hat{y} = \sigma(0.434) = 0.607$$

</>TensorFlow

```
model = tf.keras.Sequential([
    tf.keras.layers.Dense(
        64, activation='relu',
        input_shape=(784,)),
    tf.keras.layers.Dense(
        32, activation='relu'),
    tf.keras.layers.Dense(
        10, activation='softmax')
])
```

</>PyTorch

```
class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(784, 64)
        self.fc2 = nn.Linear(64, 32)
        self.fc3 = nn.Linear(32, 10)
    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = torch.relu(self.fc2(x))
        return torch.softmax(self.fc3(x), dim=1)
```

From Iris dataset:

Sample	Sepal L	Sepal W	Petal L
1	5.1	3.5	1.4
2	6.2	3.4	5.4

So input matrix:

$$X = \begin{bmatrix} 5.1 & 3.5 & 1.4 \\ 6.2 & 3.4 & 5.4 \end{bmatrix}$$

Shape:

$$(2 \times 3)$$

- 2 samples
- 3 features

MLP Architecture

We now build a **2-layer MLP**:

$$3 \rightarrow 2 \rightarrow 2$$

- Input layer: 3 neurons
- Hidden layer: 2 neurons
- Output layer: 2 neurons
- Activation: Sigmoid (for demonstration)

◆ Layer 1 (Input → Hidden)

Weight Matrix W_1

$$W_1 = \begin{bmatrix} 0.2 & -0.3 \\ 0.4 & 0.1 \\ -0.5 & 0.2 \end{bmatrix}$$

Shape: (3×2)

Bias:

$$b_1 = [0.1, -0.2]$$



Compute Hidden Layer

$$Z_1 = XW_1 + b_1$$

From previous calculation:

$$Z_1 = \begin{bmatrix} 1.82 & -1.10 \\ 0 & -0.64 \end{bmatrix}$$

Apply sigmoid:

$$A_1 = \sigma(Z_1)$$

$$A_1 = \begin{bmatrix} 0.86 & 0.25 \\ 0.50 & 0.35 \end{bmatrix}$$

Shape:

$$(2 \times 2)$$

This becomes input to next layer.

◆ Layer 2 (Hidden → Output)

Now we define:

Weight Matrix W_2

$$W_2 = \begin{bmatrix} 0.3 & -0.4 \\ 0.2 & 0.5 \end{bmatrix}$$

Shape: (2×2)

Bias:

$$b_2 = [0.05, 0.1]$$

◆ For Sample 1: [0.86, 0.25]

Output Neuron 1:

$$\begin{aligned}(0.86)(0.3) + (0.25)(0.2) \\ = 0.258 + 0.05 \\ = 0.308\end{aligned}$$

Add bias:

$$0.308 + 0.05 = 0.358$$

Output Neuron 2:

$$\begin{aligned}(0.86)(-0.4) + (0.25)(0.5) \\ = -0.344 + 0.125 \\ = -0.219\end{aligned}$$

Add bias:

$$-0.219 + 0.1 = -0.119$$

◆ For Sample 2: [0.50, 0.35]

Output Neuron 1:

$$\begin{aligned}(0.50)(0.3) + (0.35)(0.2) \\ = 0.15 + 0.07 \\ = 0.22\end{aligned}$$

Add bias:

$$0.22 + 0.05 = 0.27$$

Output Neuron 2:

$$\begin{aligned}(0.50)(-0.4) + (0.35)(0.5) \\ = -0.20 + 0.175 \\ = -0.025\end{aligned}$$

Add bias:

$$-0.025 + 0.1 = 0.075$$



Matrix Before Final Activation

$$Z_2 = \begin{bmatrix} 0.358 & -0.119 \\ 0.27 & 0.075 \end{bmatrix}$$



Apply Sigmoid Again

$$A_2 = \sigma(Z_2)$$

Sample 1

$$\sigma(0.358) = 0.588$$

$$\sigma(-0.119) = 0.470$$

Sample 2

$$\sigma(0.27) = 0.567$$

$$\sigma(0.075) = 0.519$$



Final Output of 2-Layer MLP

$$A_2 = \begin{bmatrix} 0.588 & 0.470 \\ 0.567 & 0.519 \end{bmatrix}$$

Shape:

(2×2)

Activation Functions: Introducing Non-Linearity



Sigmoid

Formula:

$$\sigma(x) = 1 / (1 + e^{-x})$$

Derivative:

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

Characteristics:

- Output range: **(0, 1)**
- Smooth, differentiable
- S-shaped curve
- Vanishing gradient problem

Best For:

Binary classification output layer



Tanh

Formula:

$$\tanh(x) = (e^x - e^{-x}) / (e^x + e^{-x})$$

Derivative:

$$\tanh'(x) = 1 - \tanh^2(x)$$

Characteristics:

- Output range: **(-1, 1)**
- Zero-centered
- Stronger gradients than sigmoid
- Still has vanishing gradient

Best For:

Hidden layers in shallow networks



ReLU

Formula:

$$\text{ReLU}(x) = \max(0, x)$$

Derivative:

$$\text{ReLU}'(x) = 1 \text{ if } x > 0, \text{ else } 0$$

Characteristics:

- Output range: **[0, ∞)**
- Computationally efficient
- No vanishing gradient for $x > 0$
- "Dying ReLU" problem

Best For:

Hidden layers in deep networks

Training MLPs: Optimization and Hyperparameters



Gradient Descent Variants

Batch GD

Uses entire dataset
Stable but slow

Stochastic GD

One sample at a time
Fast but noisy

Mini-batch GD

Small batches (32-256)
Best of both worlds



Learning Rate

Controls step size in weight updates:

$$w = w - \eta \times \text{gradient}$$

Too large	Diverges
Too small	Slow
Just right	Converges



Adam Optimizer

Adam (Adaptive Moment Estimation) combines momentum and adaptive learning rates for faster convergence.

Update Rules:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)g_t$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2)g_t^2$$

$$\hat{m}_t = m_t / (1-\beta_1^t)$$

$$\hat{v}_t = v_t / (1-\beta_2^t)$$

$$\theta_t = \theta_{t-1} - \eta \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$$

Default: $\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=10^{-8}$



Vanishing Gradient

Gradients become extremely small in deep networks, preventing learning in early layers.

Causes:

Sigmoid/tanh saturation, deep networks

Solutions:

ReLU, Batch Norm, Residual connections



Regularization

L2 Regularization

$L = L_0 + \lambda \sum w^2$
Penalizes large weights

Dropout

Randomly drop neurons during training
 $p = 0.2-0.5$ (drop probability)

Early Stopping

Stop when validation loss increases



Key Hyperparameters

Learning Rate	0.001 - 0.1
Batch Size	16 - 512
Epochs	10 - 1000+
Hidden Layers	1 - 10+
Neurons/Layer	32 - 2048

CHAPTER 03

Convolutional Neural Networks

Revolutionizing Computer Vision: Convolution, Pooling, and Feature Learning



Convolution



Pooling



Feature Extraction

Convolution Operation and Feature Extraction



What is Convolution?

Convolution is a mathematical operation that slides a small matrix called a **filter** or **kernel** over the input image to extract features like edges, textures, and patterns.

2D Convolution Formula:

$$S(i,j) = (I * K)(i,j) = \sum_m \sum_n I(i+m, j+n) \cdot K(m,n)$$

I = Input, K = Kernel, S = Output feature map

Hyperparameters

Filter Size ($f \times f$)

Common: 3×3 , 5×5 , 7×7

Stride (s)

Step size: $s=1$ (overlapping), $s=2$ (non-overlapping)

Padding (p)

Same (preserve size), Valid (reduce size)

Output Size Formula:

$$O = L(n + 2p - f)/s + 1$$



Numerical Example

Input: 5×5 image, Kernel: 3×3 , Stride: 1, Padding: 0

Input Image:

```
[1 2 3 0 1]
[0 1 2 3 0]
[1 0 1 2 3]
[3 2 1 0 1]
[0 1 0 3 2]
```

Kernel (Edge Detector):

```
[ 1 0 -1]
[ 1 0 -1]
[ 1 0 -1]
```

First Position (Top-Left):

Region: [1 2 3; 0 1 2; 1 0 1]

$$\begin{aligned} \text{Sum} &= 1 \times 1 + 2 \times 0 + 3 \times (-1) + 0 \times 1 + 1 \times 0 + 2 \times (-1) + 1 \times 1 + 0 \times 0 + 1 \times (-1) \\ &= 1 + 0 - 3 + 0 + 0 - 2 + 1 + 0 - 1 = -4 \end{aligned}$$

Output: 3×3 feature map

</>TensorFlow

```
model = tf.keras.Sequential([
    tf.keras.layers.Conv2D(
        32, (3, 3), activation='relu',
        input_shape=(28, 28, 1)),
    tf.keras.layers.Conv2D(
        64, (3, 3), activation='relu'),
    tf.keras.layers.MaxPooling2D((2, 2))
])
```

</>PyTorch

```
class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 32, 3)
        self.conv2 = nn.Conv2d(32, 64, 3)
        self.pool = nn.MaxPool2d(2, 2)
    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        return x
```

Pooling Layers and Complete CNN Architecture



Pooling Layers

Pooling reduces spatial dimensions while retaining important features, providing translation invariance and reducing computation.

Max Pooling

Selects maximum value

[1 3] [3]
[2 4] → [4]

Preserves strongest features

Average Pooling

Computes average value

[1 3] [2.5]
[2 4] → [3]

Smooths features

Benefits:

- ✓ Reduces spatial dimensions by 75% (2×2 pooling)
- ✓ Provides translation invariance
- ✓ Prevents overfitting
- ✓ Reduces computational cost

Complete CNN Architecture

- I** **Input Layer**
28×28×1 (grayscale image)
- C1** **Conv + ReLU**
32 filters, 3×3 → 26×26×32
- P1** **Max Pooling**
2×2 → 13×13×32
- C2** **Conv + ReLU**
64 filters, 3×3 → 11×11×64
- P2** **Max Pooling**
2×2 → 5×5×64
- F** **Flatten + FC**



Dimension Calculation Example

Input: 28×28×1, Conv1: 32 filters, 3×3, stride=1

After Conv1:

$$O = (28 - 3) / 1 + 1 = 26$$

Output: 26×26×32

After Pool1 (2×2, stride=2):

$$O = 26 / 2 = 13$$

Output: 13×13×32

After Conv2 (64 filters, 3×3):

$$O = (13 - 3) / 1 + 1 = 11$$

Output: 11×11×64

After Pool2 (2×2, stride=2):

$$O = 11 / 2 = 5 \text{ (floor)}$$

Output: 5×5×64 = 1600 features



Feature Hierarchy

Layer 1 (Low-level)

Edges, corners, color gradients

Layer 2-3 (Mid-level)

Textures, shapes, patterns

Layer 4+ (High-level)

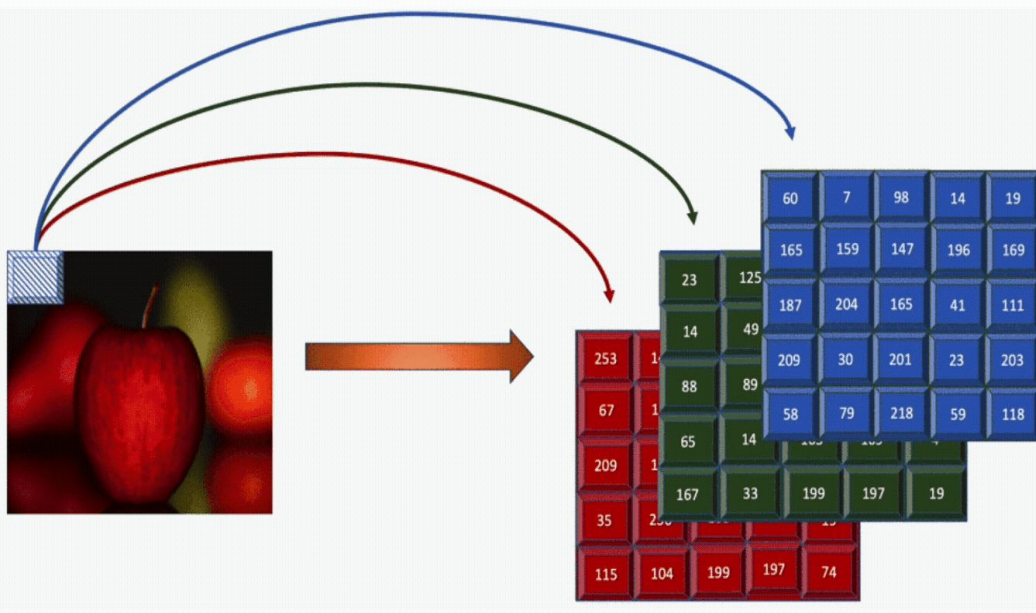
Object parts, complex features

</>TensorFlow

```
model = tf.keras.Sequential([
```

</>PyTorch

Input Image



Input Image in pixels

0	0	0	0	0	0	0
0	60	113	56	139	85	0
0	73	121	54	84	128	0
0	131	99	70	129	127	0
0	80	57	115	69	134	0
0	104	126	123	95	130	0
0	0	0	0	0	0	0

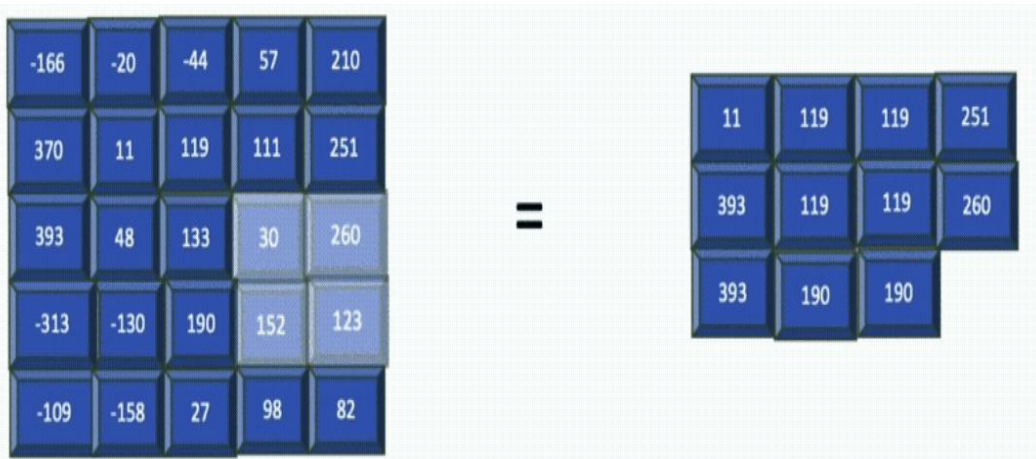
Input Image after kernel

Kernel

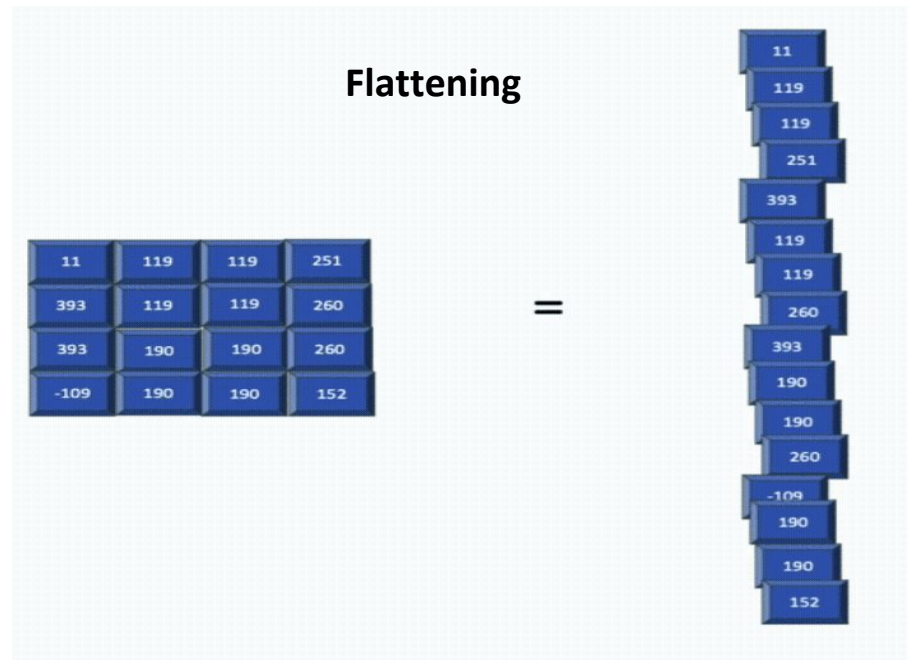
0	-1	0
-1	5	-1
0	-1	0

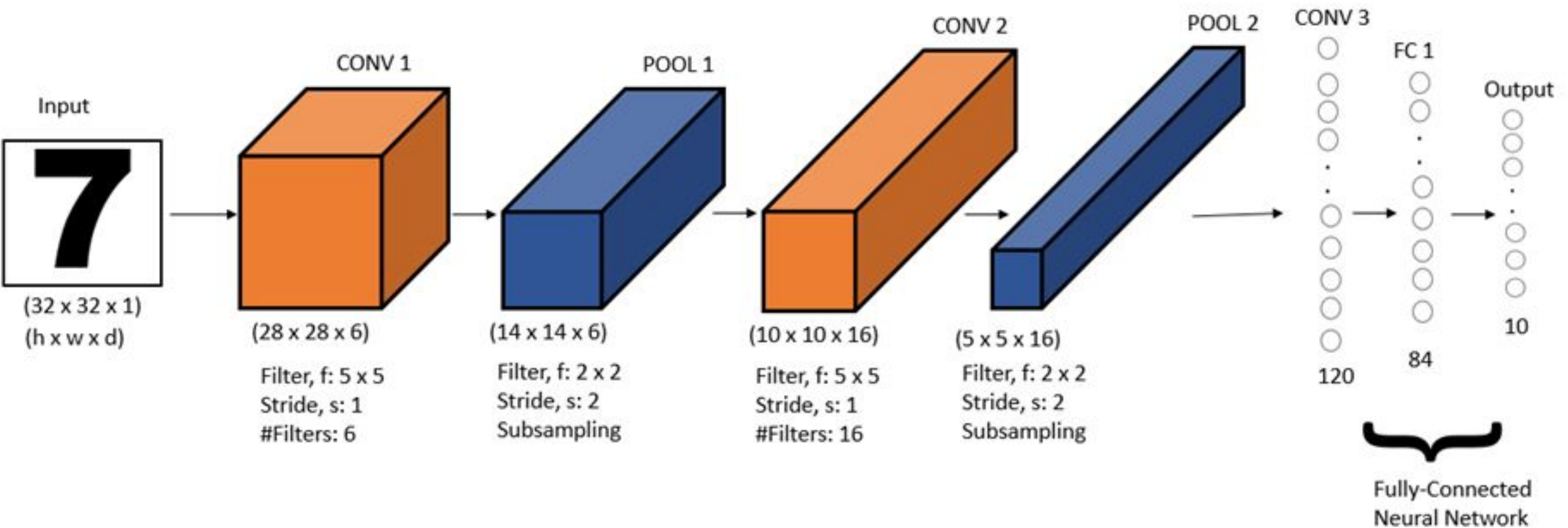
114	328	-26	470	158
53	266	-61		

Max Pooling

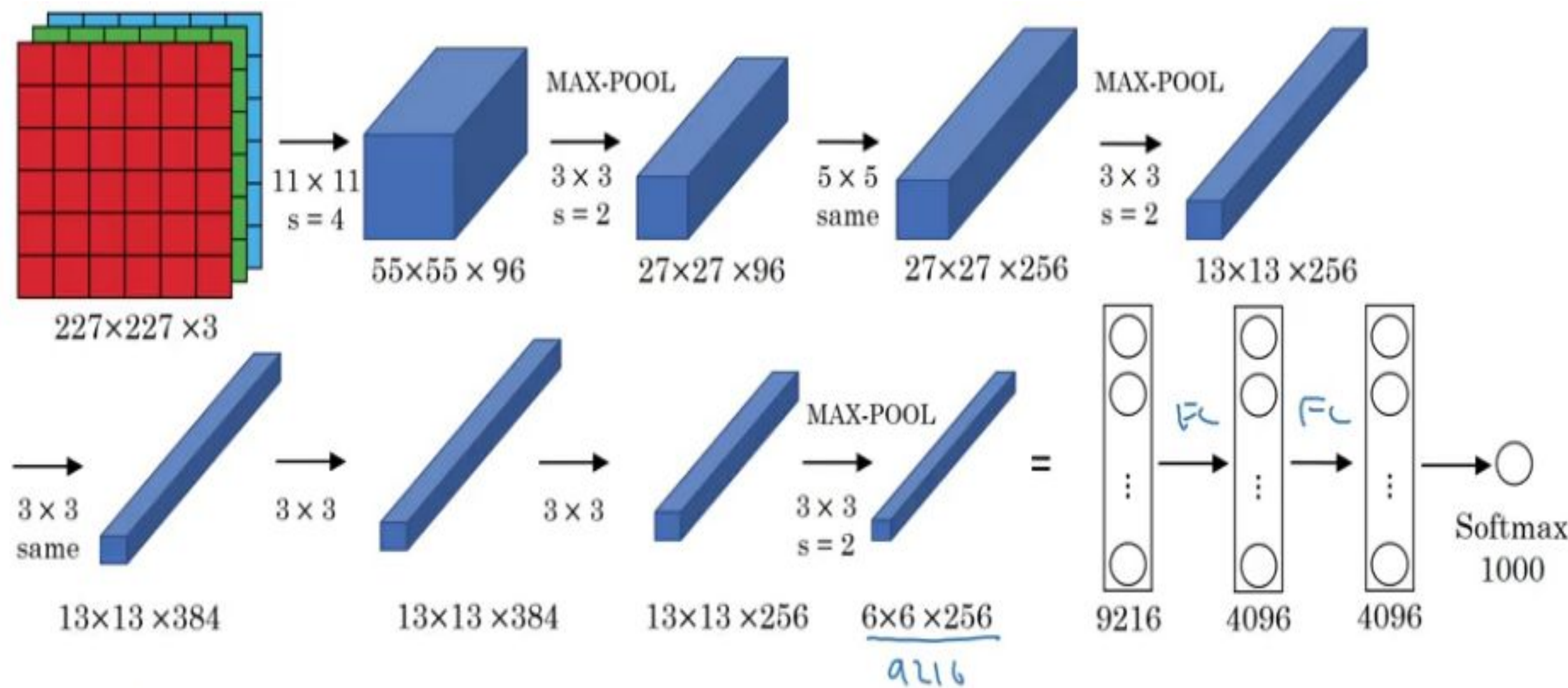


Flattening





$$\text{Output size} = \frac{(N - F)}{S} + 1$$



Code Ntbk Link:

https://drive.google.com/file/d/1S_1DEb-aeg6exkU6jbDQoC6hAGFRf73Q/view?usp=drive_link

Dataset Download Link:

https://drive.google.com/file/d/19IPh1HpN7WbOi4Be57g7RnqAmQhfXCUE/view?usp=drive_link



Question

A 5×5 input feature map is given as:

$$\begin{bmatrix} 1 & 2 & 3 & 0 & 1 \\ 4 & 5 & 6 & 1 & 2 \\ 7 & 8 & 9 & 2 & 3 \\ 1 & 0 & 1 & 3 & 4 \\ 2 & 1 & 2 & 4 & 5 \end{bmatrix}$$

A 3×3 convolution filter is applied:

$$\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

- Stride = 1
- Padding = 0 (Valid convolution)

Tasks:

1. Calculate the output feature map size.
2. Compute the output feature map values.

Step 1: Output Size Calculation

Formula:

$$\text{Output} = \frac{N + 2P - F}{S} + 1$$

Where:

- $N = 5$ (input size)
- $F = 3$ (filter size)
- $S = 1$ (stride)

$$\text{Output} = \frac{(5 - 3)}{1} + 1 = 3$$

✅ Output size = 3×3

Step 2: Convolution Operation

We slide the filter over the input and compute element-wise multiplication + sum.

◆ Position (1,1)

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Multiply with filter:

$$\begin{aligned} & (1 \times 1 + 2 \times 0 + 3 \times (-1)) + (4 \times 1 + 5 \times 0 + 6 \times (-1)) + (7 \times 1 + 8 \times 0 + 9 \times (-1)) \\ &= (1 - 3) + (4 - 6) + (7 - 9) \\ &= (-2) + (-2) + (-2) = -6 \end{aligned}$$

◆ Position (1,2)

$$\begin{bmatrix} 2 & 3 & 0 \\ 5 & 6 & 1 \\ 8 & 9 & 2 \end{bmatrix}$$

$$\begin{aligned} & (2 - 0) + (5 - 1) + (8 - 2) \\ &= 2 + 4 + 6 = 12 \end{aligned}$$

Final Output Feature Map

$$\begin{bmatrix} -6 & 12 & 12 \\ -4 & 7 & 7 \\ -2 & 0 & 0 \end{bmatrix}$$

Problem 1: Convolution Without Padding

Given:

Grayscale Image:

$$I = \begin{bmatrix} 2 & 1 & 0 & 2 \\ 3 & 1 & 1 & 0 \\ 0 & 2 & 3 & 1 \\ 1 & 0 & 2 & 2 \end{bmatrix}$$

Filter (Kernel):

$$K = \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix}$$

Conditions:

- Kernel size = 2×2
- Stride = 1
- Padding = 0
- Activation = ReLU
- No pooling

Students Must Compute:

1. Output feature map size
2. Convolution result
3. Feature map after ReLU



Problem 2: Convolution With Padding

Given:

Image:

$$I = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 1 & 0 \\ 2 & 1 & 2 \end{bmatrix}$$

Filter:

$$K = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}$$

Conditions:

- Kernel size = 2×2
- Stride = 1
- Padding = 1 (Zero padding)
- Activation = None

Students Must:

1. Show padded image
2. Calculate output size using formula
3. Compute full feature map

RNN

Problem

Suppose we have a simple **RNN** processing a sequence of 3 inputs.

Example sequence (e.g., sentiment over words):

$$x_1 = 1, \quad x_2 = 2, \quad x_3 = 3$$

We will compute the **hidden states step by step**.

Step 1: RNN Formula

The RNN hidden state is calculated as:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$$

Where

- x_t = input at time t
- h_{t-1} = previous hidden state
- W_x = input weight
- W_h = recurrent weight
- b = bias

Step 2: Given Values

Assume

$$W_x = 0.5$$

$$W_h = 0.8$$

$$b = 0$$

Initial hidden state

$$h_0 = 0$$

Activation function

$$\tanh(x)$$

Step 3: Time Step 1

Input

$$x_1 = 1$$

Calculation

$$h_1 = \tanh(0.5 \times 1 + 0.8 \times 0)$$

$$h_1 = \tanh(0.5)$$

$$h_1 = 0.462$$

Step 4: Time Step 2

Input

$$x_2 = 2$$

$$h_2 = \tanh(0.5 \times 2 + 0.8 \times 0.462)$$

$$h_2 = \tanh(1 + 0.3696)$$

$$h_2 = \tanh(1.3696)$$

$$h_2 = 0.878$$

Step 5: Time Step 3

Input

$$x_3 = 3$$

$$h_3 = \tanh(0.5 \times 3 + 0.8 \times 0.878)$$

$$h_3 = \tanh(1.5 + 0.7024)$$

$$h_3 = \tanh(2.2024)$$

$$h_3 = 0.976$$

Final Hidden States

Time Step	Input	Hidden State
t1	1	0.462
t2	2	0.878
t3	3	0.976

Interpretation

This example shows that:

- **RNN remembers previous inputs** through the hidden state.
- Each output depends on:
 - current input
 - previous memory.

Example applications:

- **Language translation**
- **Speech recognition**
- **Emotion recognition from physiological signals**
- **Time series prediction**